

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE Nº 07

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

Nº de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 10/10/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para esta práctica utilicé como base el código de iluminación proporcionado por el profesor, realizando algunas modificaciones en las fuentes de luz e integrando los modelos tridimensionales que había utilizado en prácticas anteriores.

Primero agregué los modelos del dado y del holocrón creados y texturizados en la práctica anterior, utilizando la misma imagen de logos de Star Wars. Para esto agregué las variables correspondientes a sus texturas y las cargué mediante LoadTextureA(), para posteriormente utilizarlas al dibujar cada figura. Además, agregué mi modelo del Rover con jerarquía y el Avión reemplazando los ya existentes para utilizarlos después en el reporte de práctica.

```
Texture brickTexture;
Texture dirtTexture;
Texture plainTexture;
Texture pisoTexture;
Texture AgaveTexture;
Texture dadoTexture;
Texture holocronTexture;

Model Blackhawk_M;
Model roverbase;
Model brazobase;
Model brazouno;
Model brazodos;
Model pinza;
Model frontalmasz;
Model frontalmenosz;
Model mediomasz;
Model mediomenosz;
Model traseramasz;
Model traseramenosz;
Model Avion_M;
```

Para que las figuras reaccionaran a la iluminación, realicé modificaciones en los valores de las normales de sus superficies, principalmente en el holocrón, donde definí las normales correspondientes a cada una de sus caras. También conservé las coordenadas de textura utilizadas anteriormente, para mantener la posición de los logos.

```
dadoTexture = Texture("Textures/LogosStarWars.jpeg");
dadoTexture.LoadTextureA();
holocronTexture = Texture("Textures/LogosStarWars.jpeg");
holocronTexture.LoadTextureA();
```

```
roverbase = Model();
roverbase.LoadModel("Models/roverbase.obj");
brazobase = Model();
brazobase.LoadModel("Models/brazobase.obj");
brazouno = Model();
brazouno.LoadModel("Models/brazouno.obj");
brazodos = Model();
brazodos.LoadModel("Models/brazodos.obj");
pinza = Model();
pinza.LoadModel("Models/pinza.obj");
frontalmasz = Model();
frontalmasz.LoadModel("Models/frontal+z.obj");
frontalmenosz = Model();
frontalmenosz.LoadModel("Models/frontal-z.obj");
mediomasz = Model();
mediomasz.LoadModel("Models/medio+z.obj");
mediomenosz = Model();
mediomenosz.LoadModel("Models/medio-z.obj");
traseramasz = Model();
traseramasz.LoadModel("Models/trasera+z.obj");
traseramenosz = Model();
traseramenosz.LoadModel("Models/trasera-z.obj");
Avion_M = Model();
Avion_M.LoadModel("Models/avion_mordecai.obj");
```

Posteriormente modifiqué la configuración de las luces de tipo SpotLight proporcionadas en el código original. En el caso de la linterna que sigue a la cámara, aumenté su ángulo de apertura de 5 a 15 grados, reduje su intensidad difusa de 2.0 a 1.0 y cambié su intensidad ambiental de 0.0 a 0.3, para conseguir una iluminación menos intensa, pero que abarcara un área más grande. También disminuí la intensidad ambiental de la luz verde fija de 1.0 a 0.5.

```
//luz direccional, sólo 1 y siempre debe de existir
mainLight = DirectionalLight(1.0f, 1.0f, 1.0f,
    0.3f, 0.3f,
    0.0f, 0.0f, -1.0f);
//contador de luces puntuales
unsigned int pointLightCount = 0;
//Declaración de primer luz puntual
pointLights[0] = PointLight(1.0f, 0.0f, 0.0f,
    0.0f, 1.0f,
    -6.0f, 1.5f, 1.5f,
    0.3f, 0.2f, 0.1f);
pointLightCount++;
```

```
unsigned int spotLightCount = 0;
//linterna
spotLights[0] = SpotLight(1.0f, 1.0f, 1.0f,
    0.3f, 1.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, -1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    15.0f);
spotLightCount++;

//luz fija
spotLights[1] = SpotLight(0.0f, 1.0f, 0.0f,
    0.5f, 2.0f,
    5.0f, 10.0f, 0.0f,
    0.0f, -5.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    15.0f);
spotLightCount++;
```

Para el primer ejercicio utilicé el dado texturizado por código que realicé en la práctica anterior, sustituyendo los valores proporcionados por el profesor con los de mi propio modelo. Para esto conservé los índices correspondientes a sus seis caras y modifiqué las coordenadas S y T de los vértices para colocar los logos de Star Wars en cada una de ellas.

También modifiqué los valores de las normales de la cara frontal, necesarios para determinar cómo recibe la iluminación cada superficie del dado

```
void CrearDado()
{
    unsigned int cubo_indices[] = {
        // front
        0, 1, 2,
        2, 3, 0,
        // back
        8, 9, 10,
        10, 11, 8,
        // left
        12, 13, 14,
        14, 15, 12,
        // bottom
        16, 17, 18,
        18, 19, 16,
        // top
        20, 21, 22,
        22, 23, 20,
        // right
        4, 5, 6,
        6, 7, 4,
    };
    GLfloat cubo_vertices[] = {
        // front
        //x     y       z       S       T           NX      NY      NZ
        -0.5f, -0.5f,  0.5f,    0.05f,  0.76f,      0.0f,   0.0f,   -1.0f,  //0
        0.5f, -0.5f,  0.5f,     0.24f,  0.76f,      0.0f,   0.0f,   -1.0f,  //1
        0.5f,  0.5f,  0.5f,     0.24f,  0.98f,      0.0f,   0.0f,   -1.0f,  //2
        -0.5f,  0.5f,  0.5f,    0.05f,  0.98f,      0.0f,   0.0f,   -1.0f,  //3
```

```
    // right
    //x     y       z       S       T
    0.5f, -0.5f,  0.5f,     0.02f,  0.02f,      -1.0f,  0.0f,   0.0f,
    0.5f, -0.5f,  -0.5f,    0.24f,  0.02f,      -1.0f,  0.0f,   0.0f,
    0.5f,  0.5f,  -0.5f,    0.24f,  0.26f,      -1.0f,  0.0f,   0.0f,
    0.5f,  0.5f,  0.5f,     0.02f,  0.26f,      -1.0f,  0.0f,   0.0f,
    // back
    -0.5f, -0.5f, -0.5f,    0.28f,  0.26f,      0.0f,   0.0f,   1.0f,
    0.5f, -0.5f, -0.5f,     0.48f,  0.26f,      0.0f,   0.0f,   1.0f,
    0.5f,  0.5f, -0.5f,     0.48f,  0.5f,       0.0f,   0.0f,   1.0f,
    -0.5f,  0.5f, -0.5f,    0.28f,  0.5f,       0.0f,   0.0f,   1.0f,
    // left
    //x     y       z       S       T
    -0.5f, -0.5f,  -0.5f,   0.3f,  0.76f,       1.0f,   0.0f,   0.0f,
    -0.5f, -0.5f,  0.5f,    0.47f,  0.76f,      1.0f,   0.0f,   0.0f,
    -0.5f,  0.5f,  0.5f,    0.47f,  0.97f,      1.0f,   0.0f,   0.0f,
    -0.5f,  0.5f,  -0.5f,   0.3f,   0.97f,      1.0f,   0.0f,   0.0f,
    // bottom
    //x     y       z       S       T
    -0.5f, -0.5f,  0.5f,    0.04f,  0.27f,      0.0f,   1.0f,   0.0f,
    0.5f,  -0.5f,  0.5f,    0.22f,  0.27f,      0.0f,   1.0f,   0.0f,
     0.5f,  -0.5f,  -0.5f,  0.22f,  0.49f,      0.0f,   1.0f,   0.0f,
    -0.5f, -0.5f,  -0.5f,   0.04f,  0.49f,      0.0f,   1.0f,   0.0f,
    //UP
    //x       y       z       S       T
     -0.5f, 0.5f,  0.5f,    0.14f,  0.51f,      0.0f,   -1.0f,  0.0f,
     0.5f,  0.5f,  0.5f,    0.31f,  0.51f,      0.0f,   -1.0f,  0.0f,
      0.5f, 0.5f,  -0.5f,   0.31f,  0.72f,      0.0f,   -1.0f,  0.0f,
     -0.5f, 0.5f,  -0.5f,   0.14f,  0.72f,      0.0f,   -1.0f,  0.0f,
};
MeshModel* dado = new MeshModel();
dado->CreateMeshModel(cubo_vertices, cubo_indices, 192, 36);
meshListModel.push_back(dado);
```

Para el segundo ejercicio sustituí el holocrón del profesor por el modelo que creé y texturicé por código en la práctica anterior, conservando su forma y las coordenadas de textura correspondientes a cada uno de los logos.

```
void CrearHolocron()
{
    unsigned int holocron_indices[] = {
        // Frente
        0, 1, 2,
        2, 3, 0,
        // Lateral derecho
        4, 5, 6,
        6, 7, 4,
        // Parte posterior
        8, 9, 10,
        10, 11, 8,
        // Lateral izquierdo
        12, 13, 14,
        14, 15, 12,
        // Base inferior
        16, 17, 18,
        18, 19, 16,
        // Base superior
        20, 21, 22,
        22, 23, 20,
        // Esquinas superiores
        24, 25, 26,
        27, 28, 29,
        // Esquinas inferiores
        30, 31, 32,
        33, 34, 35,
        // Esquinas posteriores superiores
        36, 37, 38,
        39, 40, 41,
        // Esquinas posteriores inferiores
        42, 43, 44,
        45, 46, 47,
    };
```

```
GLfloat holocron_vertices[] = {
    // Frente
    // X     Y     Z       S     T       NX    NY    NZ
    -0.5f, 0.0f, 0.5f,   0.06f, 0.76f,  0.0f, 0.0f, -1.0f,
    0.0f, 0.5f, 0.5f,    0.06f, 0.96f,  0.0f, 0.0f, -1.0f,
    0.5f, 0.0f, 0.5f,    0.24f, 0.96f,  0.0f, 0.0f, -1.0f,
    0.0f, -0.5f, 0.5f,   0.24f, 0.76f,  0.0f, 0.0f, -1.0f,
    // Derecha
    0.5f, 0.0f, -0.5f,  0.3f, 0.76f,  -1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.0f,   0.3f, 0.96f,  -1.0f, 0.0f, 0.0f,
    0.5f, 0.0f, 0.5f,   0.46f, 0.96f,  -1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.0f,  0.46f, 0.76f,  -1.0f, 0.0f, 0.0f,
    // Atrás
    -0.5f, 0.0f, -0.5f, 0.54f, 0.76f,  0.0f, 0.0f, 1.0f,
    0.0f, 0.5f, -0.5f,  0.54f, 0.96f,  0.0f, 0.0f, 1.0f,
    0.5f, 0.0f, -0.5f,  0.7f, 0.96f,  0.0f, 0.0f, 1.0f,
    0.0f, -0.5f, -0.5f, 0.7f, 0.76f,  0.0f, 0.0f, 1.0f,
    // Izquierda
    -0.5f, 0.0f, 0.5f,  0.78f, 0.76f,  -1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.0f,  0.78f, 0.96f,  -1.0f, 0.0f, 0.0f,
    -0.5f, 0.0f, -0.5f, 0.94f, 0.96f,  -1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.0f, 0.94f, 0.76f,  -1.0f, 0.0f, 0.0f,
    // Inferior
    0.5f, -0.5f, 0.0f,  0.14f, 0.52f,  0.0f, 1.0f, 0.0f,
    0.0f, -0.5f, -0.5f, 0.14f, 0.72f,  0.0f, 1.0f, 0.0f,
    -0.5f, -0.5f, 0.0f, 0.3f, 0.72f,  0.0f, 1.0f, 0.0f,
    0.0f, -0.5f, 0.5f,  0.3f, 0.52f,  0.0f, 1.0f, 0.0f,
    // Superior
    0.5f, 0.5f, 0.0f,   0.53f, 0.04f,  0.0f, -1.0f, 0.0f,
    0.0f, 0.5f, -0.5f,  0.53f, 0.24f,  0.0f, -1.0f, 0.0f,
    -0.5f, 0.5f, 0.0f,  0.7f, 0.24f,  0.0f, -1.0f, 0.0f,
    0.0f, 0.5f, 0.5f,   0.7f, 0.04f,  0.0f, -1.0f, 0.0f,
```

```
    // Frente superior derecho
    0.5f, 0.0f, 0.5f,   0.56f, 0.5f,  0.577f, 0.577f, 0.577f,
    0.5f, 0.5f, 0.0f,   0.4f, 0.71f,  0.577f, 0.577f, 0.577f,
    0.0f, 0.5f, 0.5f,   0.7f, 0.71f,  0.577f, 0.577f, 0.577f,
    // Frente superior izquierdo
    -0.5f, 0.5f, 0.0f,  0.13f, 0.2f,  -0.577f, 0.577f, 0.577f,
    -0.5f, 0.0f, 0.5f,  0.0f, 0.47f,  -0.577f, 0.577f, 0.577f,
    0.0f, 0.5f, 0.5f,   0.25f, 0.47f,  -0.577f, 0.577f, 0.577f,
    // Frente inferior derecho
    0.5f, -0.5f, 0.0f,  0.64f, 0.72f,  0.577f, -0.577f, 0.577f,
    0.5f, 0.0f, 0.5f,   0.8f, 0.5f,  0.577f, -0.577f, 0.577f,
    0.0f, -0.5f, 0.5f,  0.92f, 0.72f,  0.577f, -0.577f, 0.577f,
    // Frente inferior izquierdo
    -0.5f, 0.0f, 0.5f,  0.38f, 0.22f,  -0.577f, -0.577f, 0.577f,
    -0.5f, -0.5f, 0.0f, 0.25f, 0.47f,  -0.577f, -0.577f, 0.577f,
    0.0f, -0.5f, 0.5f,  0.5f, 0.47f,  -0.577f, -0.577f, 0.577f,
    // Atrás superior derecho
    0.5f, 0.5f, 0.0f,   0.49f, 0.46f,  0.577f, 0.577f, -0.577f,
    0.5f, 0.0f, -0.5f,  0.61f, 0.26f,  0.577f, 0.577f, -0.577f,
    0.0f, 0.5f, -0.5f,  0.71f, 0.46f,  0.577f, 0.577f, -0.577f,
    // Atrás superior izquierdo
    -0.5f, 0.0f, -0.5f, 0.0f, 0.22f,  -0.577f, 0.577f, -0.577f,
    -0.5f, 0.5f, 0.0f,  0.12f, 0.0f,  -0.577f, 0.577f, -0.577f,
    0.0f, 0.5f, -0.5f,  0.25f, 0.22f,  -0.577f, 0.577f, -0.577f,
    // Atrás inferior derecho
    0.5f, 0.0f, -0.5f,  0.38f, 0.29f,  0.577f, -0.577f, -0.577f,
    0.5f, -0.5f, 0.0f,  0.51f, 0.03f,  0.577f, -0.577f, -0.577f,
    0.0f, -0.5f, -0.5f, 0.24f, 0.03f,  0.577f, -0.577f, -0.577f,
    // Atrás inferior izquierdo
    -0.5f, -0.5f, 0.0f, 0.72f, 0.28f,  -0.577f, -0.577f, -0.577f,
    -0.5f, 0.0f, -0.5f, 0.84f, 0.54f,  -0.577f, -0.577f, -0.577f,
    0.0f, -0.5f, -0.5f, 0.98f, 0.28f,  -0.577f, -0.577f, -0.577f,
};
MeshModel* holocron = new MeshModel();
holocron->CreateMeshModel(holocron_vertices, holocron_indices, 384, 60);
meshListModel.push_back(holocron);
```

Después agregué la textura utilizada anteriormente y modifiqué la escala del holocrón, cambiándola de 0.5 a 1.0 para visualizarlo con un mayor tamaño en el escenario. Con esto pude observar sus diferentes superficies y los efectos de las fuentes de iluminación al recorrerlo con la cámara.

```
//Cubo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-5.0f, 5.0f, 6.0));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dadoTexture.UseTexture();
meshListModel[4]->RenderMeshModel();

//Holocron
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-7.0f, 7.0f, 6.0));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
holocronTexture.UseTexture();
meshListModel[5]->RenderMeshModel();
```

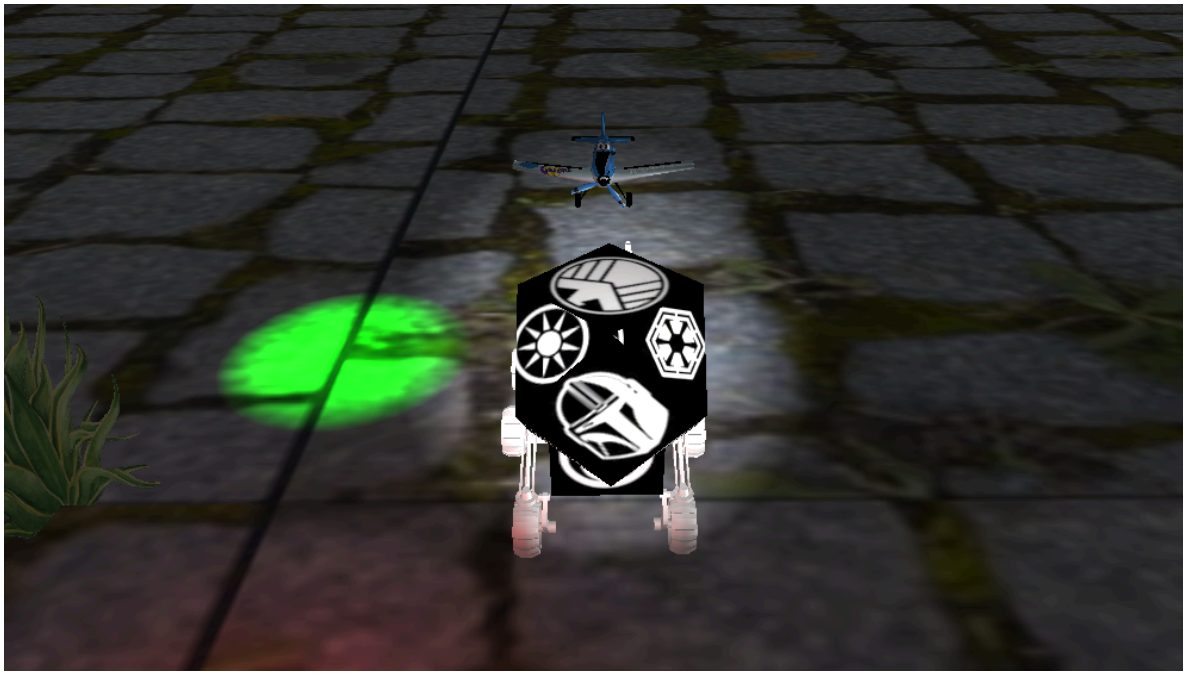
La ejecución final se muestra a continuación:



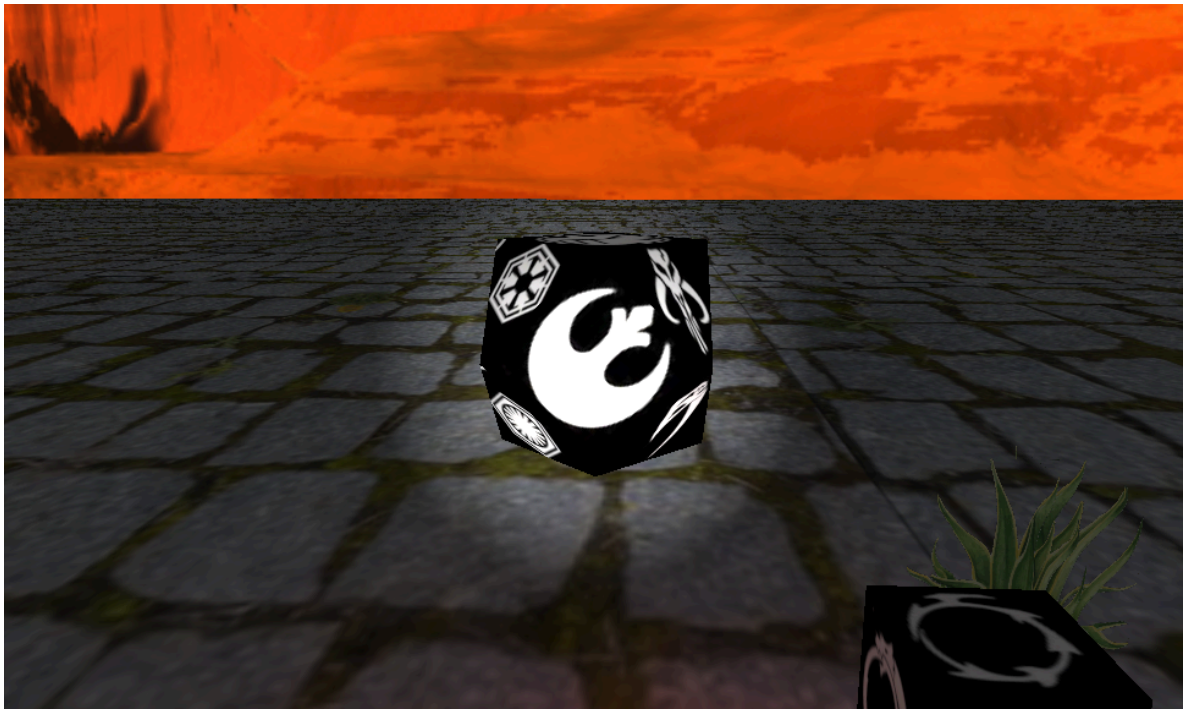


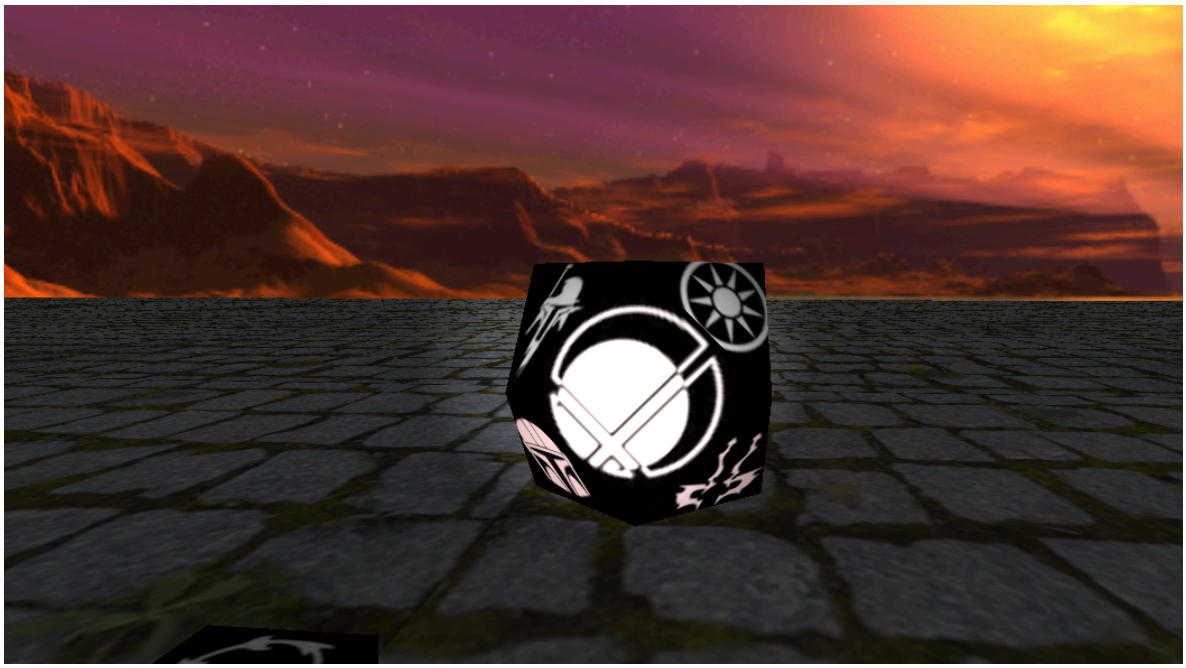


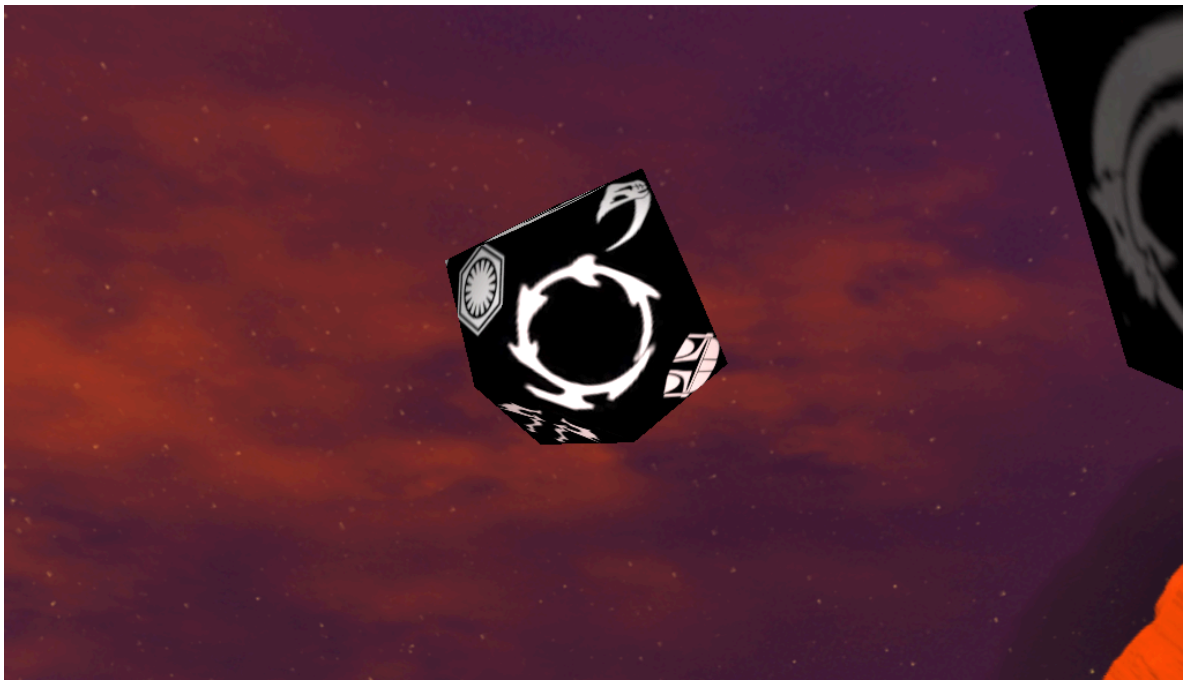












2. Problemas presentados.

No hubo ningún problema en la solución de la práctica, fue bastante rápida de realizar.

3. Conclusión:

Con la realización de este ejercicio pude comprender mejor la importancia de las normales de las superficies para calcular la iluminación de los modelos

tridimensionales en OpenGL. Al integrar el dado y el holocrón que había creado y texturizado anteriormente, pude observar cómo las diferentes fuentes de luz afectan a sus caras dependiendo de la orientación y posición de la cámara. Principalmente, el trabajo con las normales del holocrón me permitió comprender mejor cómo se calcula la iluminación en superficies con diferentes inclinaciones. Además, los ajustes realizados en las luces SpotLight me ayudaron a observar cómo influyen la intensidad y el ángulo de apertura en la iluminación del escenario.